MTB Trail Map Framework

Implementation Plan

Generated 2026-04-02

Context

Replace the existing trail map at trailmaps.app/ramba (built with Leaflet + Mapbox tiles) with a reusable framework that generates self-hosted, static trail maps from OpenStreetMap data. No external tile services, no API keys.

The first instance will be RAMBA (super-relation 12425503) in Ishpeming/Negaunee, MI, but the tool should work for any trail system with OSM data.

The existing heatmap project already uses MapLibre GL JS 5.21, PMTiles, and Docker, so this fits naturally into the toolchain.

Answers to Your Questions

Is MapBox GL JS a good viewer? MapLibre GL JS (the free, open-source fork) is better. Same rendering engine, no API key required, fully self-hostable. Already in use in the heatmap project.

Is hosting your own tiles the best idea? Yes. Protomaps makes this easy: extract the bounding box from their planet PMTiles file, serve it statically.

PMTiles via Caddy? Perfect fit. Caddy's file_server natively supports HTTP range requests, which is all PMTiles needs. No special plugin required.

Architecture

```
configs/ramba.yaml  -->  build.py  -->  build/ramba/  -->  Caddy  -->  trailmaps.app/ramba
                           |
                    +------+------+
                    v      v      v
              Overpass   pmtiles  templates/
              API        extract  (HTML/JS/CSS)
```

Input: YAML config file per trail system (relation ID, bbox, logo, parking info)

Pipeline: Python scripts fetch OSM data, compute parallel-line offsets, extract basemap tiles

Output: Static site (index.html + trails.geojson + basemap.pmtiles + assets)

Project Structure

```
mtb-map-framework/
  configs/
    ramba.yaml               # Per-map configuration
```

```
scripts/
  build.py                # Orchestrator - runs all steps
  fetch_trails.py         # Overpass API -> trails.geojson
  fetch_pois.py           # Overpass API -> pois.geojson (guideposts)
  compute_offsets.py      # Adds line-offset values for shared segments
  fetch_basemap.py        # pmtiles extract -> basemap.pmtiles
templates/
  index.html              # MapLibre GL JS viewer
  style.css               # Responsive styling
  app.js                  # Map logic, trail toggles, popups
assets/
  fonts/                  # Protomaps self-hosted glyph PBFs
  sprites/                # Protomaps sprite sheets (light/dark)
  maps/ramba/             # Logo, favicon per map
build/                    # Generated output (gitignored)
  ramba/
    index.html, trails.geojson, pois.geojson,
    basemap.pmtiles, fonts/, sprites/, logo.png, favicon.ico
requirements.txt          # requests, pyyaml
```

Config File (configs/ramba.yaml)

```yaml
name: RAMBA
slug: ramba
title: "RAMBA Trail Map"
super_relation_id: 12425503       # OSM super-relation

bbox: [-87.66, 46.48, -87.60, 46.51]  # west, south, east, north
center: [-87.635, 46.495]
zoom: 14
min_zoom: 12
max_zoom: 18
basemap_maxzoom: 15

logo: assets/maps/ramba/logo.png
favicon: assets/maps/ramba/favicon.ico

parking:
  - name: "HOB Trailhead"
    coordinates: [-87.640399, 46.4916581]
    directions_url: "https://maps.app.goo.gl/..."
  # ... more parking areas
```

To create a new map: write a new YAML config, add a logo, run `python scripts/build.py configs/newmap.yaml`.

Data Pipeline

Step 1: fetch_trails.py

- Query Overpass for super-relation to get member relation IDs + tags (name, colour)
- For each member relation, fetch member ways with `out geom` (inline coordinates)
- Output: `trails.geojson` with each way as a LineString feature containing relation_name, relation_colour, way_id
- Ways shared by multiple relations appear once per relation (enables parallel rendering)

Step 2: fetch_pois.py

- Query Overpass for guideposts (`tourism=information, information=guidepost`) in bbox
- Output: `pois.geojson` with name/ref properties

Step 3: compute_offsets.py

- Build `way_id -> set(relation_ids)` mapping
- For shared ways: assign symmetric offsets (e.g., 3 relations -> [-4, 0, 4] pixels)
- For single-use ways: offset = 0
- Adds `offset` property to each feature in trails.geojson

Step 4: fetch_basemap.py

- Runs `pmtiles extract` from Protomaps planet file for the bbox
- Produces `basemap.pmtiles` (~5-20MB for a small area)

Step 5: build.py (orchestrator)

- Parses config, runs steps 1-4, copies templates with injected CONFIG object, copies assets

---

Map Viewer

Tech Stack

- **MapLibre GL JS 5.21** - rendering (CDN with SRI hashes)
- **PMTiles JS** - loads basemap.pmtiles via range requests
- **@protomaps/basemaps** - generates basemap style layers
- Self-hosted fonts + sprites (no external requests at runtime)

Features

- **Trail layers:** Per-relation line layers with line-color from OSM colour= tag, line-offset from computed offsets, casing layer underneath
- **Trail labels:** symbol-placement: "line" with halo for readability
- **Trail selector panel:** Checkboxes per relation (colored), "Show All" toggle, collapsible on mobile
- **Parking markers:** Custom markers with popup showing name + "Get Directions" link
- **Guideposts:** Circle layer, toggleable from panel
- **Geolocation:** GeolocateControl with high-accuracy tracking
- **Logo:** Fixed-position overlay in corner
- **URL hash:** #zoom/lat/lon for sharing positions
- **Dark/light mode:** Auto-detect system preference, manual toggle, switches basemap flavor
- **Mobile-friendly:** Collapsible panel, touch-friendly controls, maxBounds prevents panning away

Parallel Line Rendering (Tube Map Style)

Where trails share a segment, MapLibre's `line-offset` paint property shifts lines perpendicular to their direction:

```
"line-offset": ["get", "offset"]  // reads pre-computed offset from GeoJSON
```

Each relation gets its own layer, so offsets create visually parallel colored lines.

Hosting (Caddy)

```
trailmaps.app {
    handle /ramba/* {
        root * /path/to/build/ramba
        uri strip_prefix /ramba
        file_server
    }
}
```

Caddy's file_server natively supports range requests. PMTiles works out of the box.

Implementation Order

Phase 1: Data Pipeline

1. `configs/ramba.yaml` - configuration
2. `scripts/fetch_trails.py` - Overpass queries + GeoJSON
3. `scripts/fetch_pois.py` - guideposts + parking from OSM
4. `scripts/compute_offsets.py` - parallel line offsets
5. `scripts/fetch_basemap.py` - PMTiles extraction
6. `scripts/build.py` - orchestrator
7. `requirements.txt`

Phase 2: Map Viewer

1. `templates/index.html` - page structure, MapLibre container, control panel
2. `templates/app.js` - map init, trail layers, toggles, parking popups, geolocation
3. `templates/style.css` - responsive layout, panel styling, dark mode

Phase 3: Assets and Integration

1. Download Protomaps fonts + sprites into assets/
2. End-to-end test: build -> serve locally -> verify all features
3. Adjust bbox, zoom, and styling based on actual RAMBA data

Phase 4: Deploy

1. Copy build output to Caddy server
2. Configure Caddy for trailmaps.app/ramba
3. Verify PMTiles range requests work in production

Verification

1. Run `python scripts/build.py configs/ramba.yaml` - should produce complete build/ramba/
2. Serve locally: `python -m http.server -d build/ramba 8080`
3. Verify: basemap tiles load, trails render with correct colors, parallel lines visible on shared segments, parking popups work with direction links, guideposts toggle on/off, geolocation works, mobile layout is usable, dark mode switches basemap + UI

Dependencies

- Python 3.9+ with `requests`, `pyyaml`
- `pmtiles` CLI (Go binary) for basemap extraction
- MapLibre GL JS 5.21 (CDN)
- PMTiles JS 4.x (CDN)
- @protomaps/basemaps 5.x (CDN)
- Protomaps basemaps-assets (fonts/sprites, downloaded once)

Key Files to Create

File	Purpose
`configs/ramba.yaml`	RAMBA trail system configuration
`scripts/build.py`	Build orchestrator
`scripts/fetch_trails.py`	Overpass API trail data extraction
`scripts/fetch_pois.py`	POI extraction (guideposts)
`scripts/compute_offsets.py`	Parallel line offset computation
`scripts/fetch_basemap.py`	PMTiles basemap extraction
`templates/index.html`	MapLibre viewer HTML
`templates/app.js`	Map JavaScript logic
`templates/style.css`	Responsive styles
`requirements.txt`	Python dependencies